

#	Business logic	Code / rule implemented	Reference
1	Credit-score source priority	TV → CV → TU → FICO10 → BEACON9 → BEACON4. The first available score becomes the application's borrower score source.	borrower_processor.py, L37–49
2	TV score bins	Cutpoints: 625, 650, 675, 700, 730, 755, 795	borrower_processor.py, L40–48
3	CV score bins	Cutpoints: 635, 650, 680, 700, 720, 745, 785	borrower_processor.py, L40–48
4	TU score bins	Cutpoints: 625, 650, 675, 700, 725, 750, 780	borrower_processor.py, L40–48
5	FICO10 score bins	Same cutpoints as TV: 625,650,675,700,730,755,795	borrower_processor.py, L40–48
6	Beacon9 score bins	Same cutpoints as CV	borrower_processor.py, L40–48
7	Beacon4 score bins	Same cutpoints as TU	borrower_processor.py, L40–48
8	Borrower eligibility	Rows without APPLICATION_NUMBER are removed.	borrower_processor.py, L72–82
9	Missing-value interpretation	" ", "*", and "" are interpreted as missing when object columns can be converted to numeric.	borrower_processor.py, L83–92
10	TU data consolidation	Phase 1 + Phase 2 TU files are concatenated; records without UNIQUE_RECORD_ID are removed.	borrower_processor.py, L97–110
11	TU special missing values	"null", "NULL", \N, "", "***" and later "*" are converted to missing. Negative numeric values are also treated as missing.	borrower_processor.py, L97–120

#	Business logic	Code / rule implemented	Reference
12	Credit score source labels	Non-null TV → credit_tv; non-null FICO10 → credit_10.	borrower_processor.py, L112–114
13	Historical CV/Beacon hit selection	A historical score is used only when its corresponding *_HIT == "hit".	borrower_processor.py, L146–180
14	Credit Vision consolidation	Combines borrower FICO8 and historical CV; for a borrower, sorts descending and keeps the highest available CRD_SCORE_CV.	borrower_processor.py, L182–217
15	Number of borrowers	NUM_OF_BOR = count(rows for APPLICATION_NUMBER) after borrower/credit merge.	borrower_processor.py, L219–250
16	Bureau naming normalization	CRD_SCORE_10 → CRD_SCORE_FICO10; CRD_SCORE_EQ → CRD_SCORE_BEACON4.	borrower_processor.py, L242–247
17	Valid credit-score range	Any bureau score <300 or >998 becomes missing.	borrower_processor.py, L252–263
18	TU dependent fields	If CRD_SCORE_TU is missing, CRD_MONTHS_TU, CRD_OPEN_TRADE_TU, and CRD_PUBLIC_DEROG_TU are also cleared.	borrower_processor.py, L264–269
19	Equifax fallback	Missing TU OPEN_TRADE, PUBLIC_DEROG, and MONTHS values are filled from the corresponding EQ fields.	borrower_processor.py, L271–276
20	Score bin assignment	np.digitize(score, cutpoints, right=False) + 1; therefore bins are numbered starting from 1.	borrower_processor.py, L280–295

#	Business logic	Code / rule implemented	Reference
21	Final borrower score selection	Iterates through the defined priority and takes the first non-null bureau score. Also records which_score.	borrower_processor.py, L297–319
22	Application-level numeric aggregation	Numeric borrower variables initially aggregate using application-level max().	borrower_processor.py, L324–334
23	Top-two TV scores	For each application, highest TV score becomes TV_Credit_Score_SMI3.0_B1; second-highest becomes ...B2.	borrower_processor.py, L336–364
24	Application score-source selection	Applications are sorted by ordered which_score, so the highest-priority source represented among borrowers is retained.	borrower_processor.py, L370–376
25	Application score statistics	Produces configurable min, max, avg, median, and weighted-average score statistics.	borrower_processor.py, L380–418
26	Weighted-average weight	Primary weight is BASE_SALARY; if missing, use TOTAL_INCOME; if weight column unavailable, use equal weight 1.0.	borrower_processor.py, L433–439
27	Score rounding	Average/median/weighted average are rounded using floor(x + 0.5) ("half up").	borrower_processor.py, L69–70 and L399–417

#	Business logic	Code / rule implemented	Reference
28	Funded statuses	Funded policy statuses: C, L, R, M, T.	policy_processor.py, L56–60
29	LRU+ funded-equivalent statuses	LRU+ statuses X, A, E can be treated as funded under the LRU+ rule.	policy_processor.py, L56–60

#	Business logic	Code / rule implemented	Reference
30	Declined status	D = declined.	policy_processor.py, L56–60
31	Policy date repair	If EFFECT date is invalid but RECEIVED is valid, copy RECEIVED → EFFECT; inverse logic for missing RECEIVED.	policy_processor.py, L205–227
32	Book year	EFFECT_SID = EFFECT_DAY_SID // 100; BOOKYEAR = EFFECT_SID // 100.	policy_processor.py, L225–227
33	One record per application	Policy duplicates are reduced using keep="last".	policy_processor.py, L229–238
34	Claim data exclusion	Claim/delinquency records from year 2020 are excluded.	policy_processor.py, L240–272
35	Claim aggregation	At application level: DELQ90_IND=max, CLAIM_IND=max, TOTAL_INCURED_LOSS=sum.	policy_processor.py, L263–272
36	MI population period	Include when EFFECT or RECEIVED month is between 200900 and 202605.	policy_processor.py, L275–289
37	MI business population	Must have LOB_CODE == "MI" and LRUPLUS_FLAG != "Y"; output LOB becomes "MI".	policy_processor.py, L275–295
38	LRU Rule 1	For pre-validation cases with a Bulk reference: LRU status must be X/A/E/D and bulk policy must be funded (C/L/R/M/T).	policy_processor.py, L371–396
39	LRU Rule 2 old	With no Bulk reference, LRU application itself must have a funded status.	policy_processor.py, L398–403
40	LRU overall period	LRU must have EFFECT or RECEIVED month in 200901–202605.	policy_processor.py, L385–413

#	Business logic	Code / rule implemented	Reference
41	Additional LRU claim population	Claim applications tied to Bulk references can be added back for EFFECT_SID 200901–202312; duplicates keep last.	policy_processor.py, L419–444
42	Collection indicator	CRA_COLLECTION = Y when either satisfied or unsatisfied collection indicator is Y; otherwise N.	policy_processor.py, L449–455
43	Judgment indicator	CRA_JUDGEMENT = Y when satisfied or unsatisfied judgment indicator is Y; otherwise N.	policy_processor.py, L457–461
44	Missing condo fee	Missing/non-numeric PRK_CONDO_FEE becomes 0.	policy_processor.py, L463–464
45	Final status = Funded	Funded when status is C/L/R/M/T, or when status is X/A/E and LOB_RENAMED == LRUPLUS.	policy_processor.py, L466–477
46	Final status = Declined	Status D becomes "Declined"; anything outside funded/declined rules becomes "Others".	policy_processor.py, L466–477
47	Recent DSR replacement	For selected populations, original TDSR/GDSR are preserved, then TDSR = PRD_COMP_TDSR, GDSR = PRD_COMP_GDSR.	policy_processor.py, L481–497
48	Event category	Claim+Delq → Delq_Claim; claim only → Claim_Only; delinquency only → Delq_Only; neither → No_Event. Declined/Others are preserved as their own categories.	policy_processor.py, L502–519
49	Binary target	No_Event → 0; Others → NaN; everything else → 1. Therefore Declined, Claim, Delq and Delq+Claim all get target=1 as written.	policy_processor.py, L520–527
50	HB100 classification	ACTLTV > 97 → PRODUCT_GROUP="HB100".	policy_processor.py, L531–558

#	Business logic	Code / rule implemented	Reference
51	Product mappings	ALAU/ALAM→ALTA; HELF/HELP→HELOC; BDWN→BORROWED DOWN; CHBK→CASHBACK; VACP→VACATION/SECOND; FE01 or foreign borrower → NEW TO CAN; CPSI/CPPI→CREDIT_ASSIST; otherwise REGULAR.	policy_processor.py, L531–558
52	Product regrouping	CASHBACK, CREDIT_ASSIST, HB100, HELOC → EXPIRED; VACATION/SECOND and NEW TO CAN → REGULAR.	policy_processor.py, L559–567
53	Valid model product population	Exclude ALTA and EXPIRED; require PRD_GOVТ_GUARANTEЕ == "Y".	policy_processor.py, L570–577
54	Refinance balance exclusion	Remove application when PRD_LOAN_PURPOSE == "R" and PRD_OUTSTANDING_BAL > 3000.	policy_processor.py, L614–632
55	Property-age imputation	If missing and AVM year is valid, PRK_PROPERTY_AGE = BOOKYEAR - AVM_YEAR_BUILT.	policy_processor.py, L634–650
56	New construction age	If PRK_CONS_TYPE == "N", force property age to 1, overriding prior result.	policy_processor.py, L651–656
57	Rural postal rule	Second postal-code character "0" → RURAL_IND="Y"; otherwise "N".	policy_processor.py, L658–670
58	New Brunswick rural override	For NB, every city except FREDERICTON, SAINT JOHN, and MONCTON is forced to rural.	policy_processor.py, L28 and L667–669
59	Economic region mapping	Province maps to economic region; unmapped provinces default to "Territories".	policy_processor.py, L42–52 and L688–700

#	Business logic	Code / rule implemented	Reference
60	Toronto region override	Ontario + postal code beginning with a Toronto FSA → "Toronto"; remaining Ontario → "Rest_of_Ontario".	policy_processor.py, L688–714
61	Historical vs future economics	Historical economics used through 202211; future/scenario economics from 202212 onward.	policy_processor.py, L716–727
62	Economic look-forward features	Creates 1–4 year shifted unemployment/HPI measures. HPI_{+n} = future HPI/current HPI; UE_{+4} is later converted to future-minus-current UE.	policy_processor.py, L743–750
63	Contract-rate fallback	Missing PRD_CONTRACT_RATE → PRD_QUAL_INT_RATE.	policy_processor.py, L776–783
64	Insurance-in-force fallback	Missing IIF → $PRK_PURCHASE_PRICE \times ACTLTV/100$; if unavailable → PRD_BASE_LOAN_AMT.	policy_processor.py, L785–812
65	Property-value fallback	Existing property value retained only if >1; otherwise use purchase price, then declared value.	policy_processor.py, L814–842
66	LGD base scenario	LGD calculator is run with configured constant HPI_change and configured EFFECT_SID.	policy_processor.py, L168–182
67	LGD backtest/stress value	LGD_BT is generated by calling LGD calculator using HPI_{+4} as HPI_change.	policy_processor.py, L184–195

#	Business logic	Code / rule implemented	Reference
68	DSR difference	DSR_DIFF = abs(TDSR - GDSR), rounded to 3 decimals.	utils.py, L260–271
69	DSR bins	$\leq 5 \rightarrow 0$, $(5, 10] \rightarrow 1$, $(10, 15] \rightarrow 2$, $(15, 20] \rightarrow 3$, $> 20 \rightarrow 4$.	utils.py, L272–279
70	ACTLTV bin	Starts at 0, uses externally supplied cutpoints, and adds ∞ as final upper bound.	utils.py, L286–305
71	Number of units recoding	Property with 3 or 4 units $\rightarrow$ category "3-4"; other numeric values become their integer string representation.	utils.py, L308–323
72	HPI transformation	HPI_+4 is clipped to non-negative values and transformed using log1p.	utils.py, L244–255

#	Business logic	Code / rule implemented	Reference
73	Borrower count bin	NUM_OF_BOR: $(0, 1] \rightarrow 1$, $(1, 2] \rightarrow 2$, $> 2 \rightarrow 3$.	preprocess.py, L526–531
74	Loan purpose binary	PRD_LOAN_PURPOSE == "R" $\rightarrow$ 1; otherwise 0.	preprocess.py, L533–534
75	Mandatory base fields	Before model populations, rows missing bankruptcy/foreclosure indicator, credit-score bin, market code, property age, port, or borrower count are removed.	preprocess.py, L536–542
76	Training vintage	Training data = BOOKYEAR 2009–2015.	preprocess.py, L546–550
77	Training status	Training includes only STATUS == "Funded".	preprocess.py, L549–550

#	Business logic	Code / rule implemented	Reference
78	Training population filters	Product/government-guarantee filter and refinance-OSB rule are applied; feature clipping enabled; COMP DSR disabled.	preprocess.py, L552–555
79	Training DSR completeness	Rows missing TDSR or GDSR after processing are removed.	preprocess.py, L562–564
80	Testing period	Testing population = EFFECT_SID 202101–202312.	preprocess.py, L579–583
81	Testing economic scenario freeze	Every testing application gets HPI_+4 and UE_+4 from the 202212 observation for its ECON_REGION .	preprocess.py, L585–595
82	Testing interest-rate assumption	Testing PRD_CONTRACT_RATE is replaced by command-line fixed_contract_rate.	preprocess.py, L594–610
83	Testing DSR logic	Model filters applied; clipping enabled; COMP TDSR/GDSR are used.	preprocess.py, L613–621
84	Testing DSR completeness	Missing TDSR/GDSR rows are dropped.	preprocess.py, L624–626
85	Recent population	Recent population requires EFFECT_SID 202510–202605 and which_score == "TV".	preprocess.py, L641–646
86	Recent economic scenario freeze	Like testing, recent records receive HPI/UE values from the application's ECON_REGION at reference month 202212.	preprocess.py, L649–660
87	Recent interest rate	Recent records also use the configured fixed contract rate.	preprocess.py, L659–675

Feature	Minimum	Maximum
POD_INSURANCE_IN_FORCE	10,000	2,700,000

Feature	Minimum	Maximum
PRK_PROPERTY_VALUE	0	3,000,000
ACTLTV	0	100
PRK_PROPERTY_AGE	0	150
PRD_LOAN_TERM	1	120
TDSR	0	48
GDSR	0	42
TOTAL_INCOME	0	450,000
PRD_AMORTIZATION	1	480
PRD_QUAL_INT_RATE	0.5	15

Issue	Why it matters	Reference
Invalid policy assignment syntax	policy["RECEIVED_DAY_SID"] / policy["EFFECT_DAY_SID"] is invalid Python syntax, so the file as uploaded does not parse.	policy_processor.py, L220–224
LGD typo/operator error	self/hpi_change appears instead of self.hpi_change.	policy_processor.py, L178–180
LRU "new period" condition conflicts with comment	Comment says EFFECT_SID > 202312 , but validation_period is coded with .le(202312). Therefore rule2_new does not implement what the comment says.	policy_processor.py, L405–413
MI filter has indexing syntax problem	policy[in_period, & ...] contains an extra comma.	policy_processor.py, L280–289

Issue	Why it matters	Reference
LRU extra-period masks appear based on wrong dataframe	extra_pre_validation_period / extra_in_period are calculated from df_lru but then used while indexing policy, which can cause index-alignment problems.	policy_processor.py, L423–437
Toronto FSA function references wrong variable	Function accepts toronto_fsa_df but uses df.loc[...].	policy_processor.py, L672–686
EFFECT_SID typo	Missing-value repair writes to "FFECT_SID" rather than "EFFECT_SID".	policy_processor.py, L768–774
Target treats Declined as event = 1	Since only Others and No_Event get special values, "Declined" falls into default=1. This may or may not be intended.	policy_processor.py, L509–527
rule2_new may never represent post-202312 records	Because its shared condition is coded <=202312, the "all statuses after 202312" business rule appears broken.	policy_processor.py, L380–413
preprocess.py contains indentation error	The uploaded file has an unexpected indent around the testing logger section, preventing normal parsing as written.	preprocess.py, around L608–611
policy_processor.py also has indentation errors	In addition to the syntax defects, the uploaded file fails Python parsing around LGD indentation.	policy_processor.py, around L184–203